COURS · LICENCE

Semestre 1

# Big Data

## AVEC PYTHON

Avec évaluation de compétences
et TPs


COURS ÉLABORÉ PAR

**Dr. Ing. Takwa Ben Aïcha Gader**

Docteur en Informatique . Ambassadrice NVIDIA

takwa.ben.aichaa@gmail.com


DONNÉES

À GRANDE ÉCHELLE

**HADOOP / MAPREDUCE / SPARK**

NoSQL / PYTHON

# Introduction au Big Data

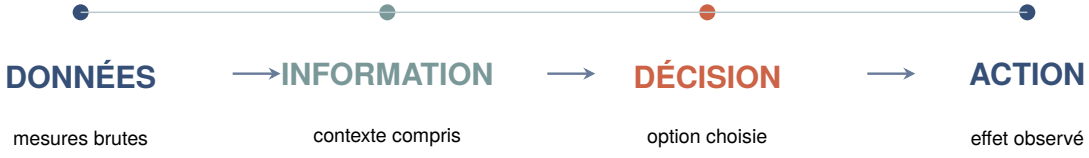


CHAPITRE 1

# Quand les données deviennent une décision



**Une donnée n'a de valeur que si elle éclaire une action au bon moment.**




**Que vaut une donnée correcte si elle arrive trop tard ?**

# Les applications sont visibles ; leur infrastructure l'est moins



DATA ENGINEER · DATA ARCHITECT

**Derrière chaque service fiable, une même chaîne de responsabilités.**

**Industrie**

machines connectées

**Finance**

paiements et fraude

**Santé**

signaux et alertes

**Mobilité**

véhicules connectés

Collecter → Transporter → Stocker → Traiter → Fiabiliser → Servir

**Les applications se voient ; leur infrastructure de données reste souvent invisible.**

## Votre objectif dans ce chapitre



**À la fin de ce chapitre, définir le Big Data ne suffira pas.**

# Défendre ou rejeter une architecture

à partir de contraintes mesurées

**MESURER** → **COMPARER** → **CHOISIR** → **TESTER** → **JUSTIFIER**

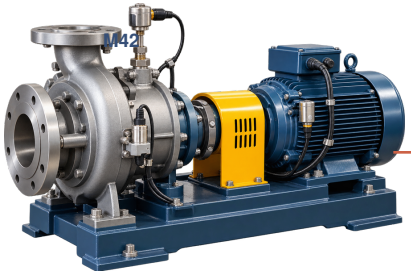
**Raisonner comme un Data Engineer junior : mesurer avant de nommer une technologie.**

# Étape 1 — 02 h 17, M42 chauffe



FIL ROUGE · PARC INDUSTRIEL IoT

**Une température grimpe. Cinq secondes pour produire une alerte utile.**




**87 °C**

**température en hausse**

vibration +18 %

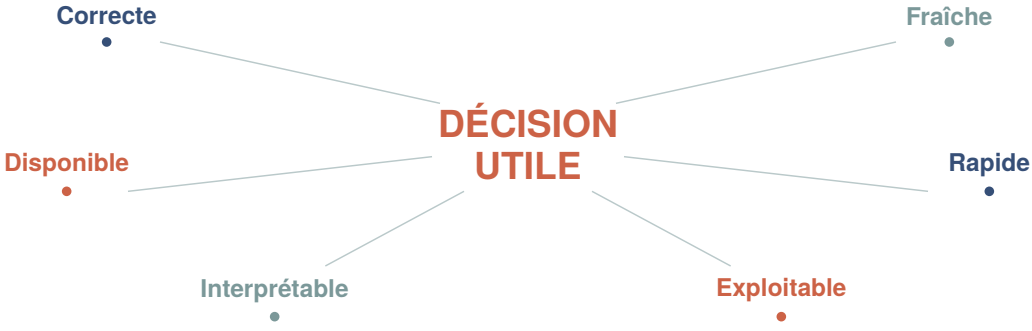
ALERTE À EXPLIQUER

**Le problème n'est pas « avoir des données » : c'est décider assez vite, avec assez de confiance.**

# Que faut-il garantir pour rendre la donnée utile ?



**Une mesure ne devient décisionnelle que si plusieurs conditions tiennent ensemble.**




**Une seule condition critique défaillante peut rendre la donnée inutilisable pour l'action.**

# Anomalie, alerte, panne : trois événements différents



**Une valeur inhabituelle ne signifie ni panne certaine, ni arrêt automatique.**

02 :17 :00 — **Anomalie** — écart au comportement attendu

02 :17 :03 — **Alerte** — demande de vérification

plus tard — **Panne possible** — fonction non assurée

**Détecter n'est pas décider : l'alerte demande une vérification, pas un arrêt automatique.**

# Que doit décider l'opératrice ?



**Une alerte utile transforme des mesures en options d'action compréhensibles.**

Contexte : tendance, vibration, charge, maintenance et fraîcheur des données.

| | |
|---|---|
| **ARRÊTER** | risque élevé confirmé ; sécuriser la machine |
| **SURVEILLER** | incertitude restante ; augmenter les contrôles |
| **POURSUIVRE** | signal expliqué ; fonctionnement jugé acceptable |

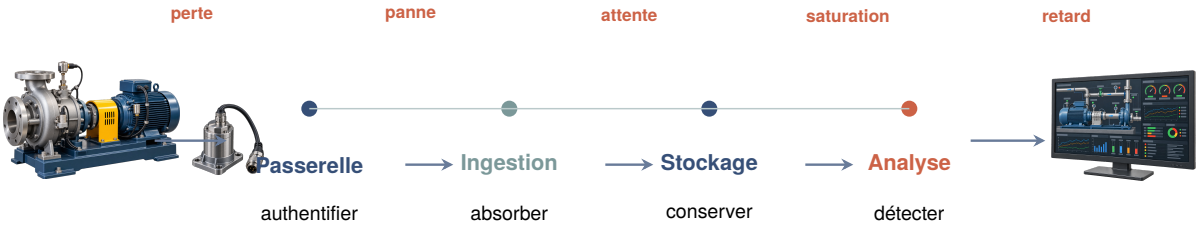
**La valeur d'une donnée dépend de la décision qu'elle améliore.**

# Du capteur à l'écran : où passent les données ?



SUIVRE UN MESSAGE

**Chaque maillon peut ralentir, perdre ou déformer l'information.**




**La latence de bout en bout additionne collecte, transport, attente, calcul et affichage.**

# Calcul guidé — combien de données arrivent ?



CALCUL GUIDÉ · UNITÉS OBLIGATOIRES

**On dimensionne avec des unités, pas avec l'impression que « cela fait beaucoup ».**

| 10 000 | 10 | 500 |
|---|---|---|
| capteurs | messages/s par capteur | octets par message |

$$10\,000 \times 10 \times 500$$

**50 Mo/s**

100 000 événements/s

**50 Mo/s décrit une contrainte ; ce nombre seul ne suffit pas à définir le Big Data.**

# Calcul guidé — combien faut-il réellement stocker ?



**La rétention brute n'est que le début du dimensionnement.**

1. $50\ \text{Mo/s} \times 86\,400$ → **4,32 To/j**
2. $4{,}32 \times 30$ → **129,6 To / 30 j**
3. $129{,}6 \times 3$ → **388,8 To répliqués**
4. $388{,}8 \times 1{,}25$ → **486 To planifiés**

L'échelle apparaît lorsque débit × temps × résilience × marge s'accumulent.

# Débit et latence : deux questions différentes



**Transporter beaucoup par seconde ne garantit pas qu'un événement arrive rapidement.**

## Débit

Combien par seconde ?

**50 Mo/s**

quantité traitée pendant une durée

## Latence

En combien de temps ?

$p95 = 3{,}8$ **s**

95 % des alertes avant 3,8 s

**Un système peut avoir un excellent débit et une mauvaise latence.**

# Activité — quel risque testez-vous en premier ?



ACTIVITÉ · VOTE · JUSTIFICATION

**Serveur candidat : 20 To, 70 Mo/s mesurés, un seul nœud. Votez avant le verdict.**

**Choisissez et justifiez.**

**A** **Capacité**
30 jours tiennent-ils ?

**B** **Débit**
le flux est-il absorbé ?

**C** **Latence**
le p95 reste-t-il sous 5 s ?

**D** **Panne**
que devient le service si le nœud tombe ?

# Verdict — une contrainte critique suffit à dire non



**Comparer chaque besoin à une mesure ou à un test reproductible.**

| Contrainte | Besoin | Candidat | Verdict |
|---|---|---|---|
| **Stockage** | 486 To planifiés | 20 To | **NON** |
| **Débit** | 65 Mo/s avec marge | 70 Mo/s | **OUI** |
| **Latence** | $p95 \leq 5$ s | non testée | **À TESTER** |
| **Disponibilité** | perdre 1 nœud | nœud unique | **NON** |

**Une architecture se défend par des preuves : capacité, charge, délai, panne et coût.**

# Les 5 V sortent du cas M42



**Les « V » nomment des difficultés observables ; ils ne remplacent pas le dimensionnement.**

| 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|
| **Volume** | **Vélocité** | **Variété** | **Véracité** | **Valeur** |
| | | | | |
| 486 To | 50 Mo/s<br>$p95 < 5$ s | mesures · logs<br>maintenance · ordres | absence<br>retard · dérive | décision<br>effet mesuré |

**Les V décrivent des contraintes ; ils ne remplacent pas le dimensionnement.**

# Même raisonnement, décisions différentes



**Le domaine change, mais on part toujours de l'action, du délai et du coût de l'erreur.**

**Santé**

prioriser un examen

Délai : minutes

Risque : alerte manquée

**Énergie**

inspecter une turbine

Délai : heures

Risque : arrêt inutile

**Paiement**

bloquer ou vérifier

Délai : secondes

Risque : client ou fraudeur lésé

**Une architecture n'est jamais rapide ou fiable dans l'absolu : elle répond à un besoin.**

# Qualité — absence, zéro et retard ne racontent pas la même chose



**Transformer une valeur sans préserver son sens peut fabriquer une fausse panne.**

### ABSENTE

—

capteur muet

DIAGNOSTIQUER

### ZÉRO

0 °C

mesure possible

VALIDER

### RETARDÉE

6 °C · 01 :32

information périmée

ÉCARTER DU TEMPS RÉEL

**Absence ≠ zéro ≠ donnée ancienne : une mesure correcte mais périmée peut être inutile.**

# Activité — trouvez la fausse alerte



90 SECONDES · EN BINÔME

**Règle naïve : « température $< 10$ °C ou $> 80$ °C = arrêter M42 ».**

DÉCISION SOUS INCERTITUDE

| | MESURE | HEURE | QUALITÉ |
|---|---|---|---|
| A | — | 02:17:01 | capteur silencieux |
| B | 87 °C | 02:17:02 | fraîche et complète |
| C | 6 °C | 01:32:00 | 45 min de retard |

**Justifier chaque réponse par le sens, la complétude et la fraîcheur de la mesure.**

# Premier levier — renforcer la machine



**Le scale-up ajoute CPU, mémoire ou disques au même serveur.**

| | | | |
|---|---|---|---|
| **CPU** | 32 cœurs | → | **64 cœurs** |
| **Mémoire** | 256 Go | → | **512 Go** |
| **Stockage** | 20 To | → | **80 To** |

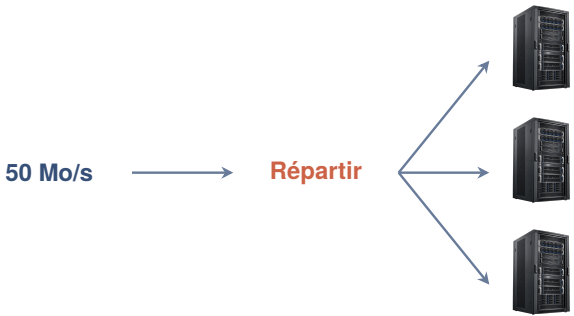
Atout : exploitation simple. Limite : plafond matériel et point de panne unique.

**Toujours tester d'abord la solution la plus simple qui satisfait réellement les contraintes.**

# Deuxième levier — répartir la charge



**Le scale-out ajoute des nœuds et répartit données ou calculs entre eux.**




**Ce qui apparaît**

- réseau et files
- coordination et synchronisation
- pannes partielles

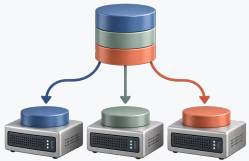
**Distribuer augmente la capacité, mais augmente aussi la complexité.**

# Partitionner n'est pas répliquer



**L'un partage le volume ; l'autre place plusieurs copies pour résister à certaines pannes.**

## Partitionner



A, B et C sont des parties différentes réparties entre les nœuds.

## Répliquer

La même donnée A existe sur plusieurs nœuds distincts.

**La réplication améliore la redondance ; elle ne remplace ni sauvegarde ni historique protégé.**

# Deux horloges — batch et streaming



**Le mode de traitement dépend du délai acceptable et du résultat attendu.**

**BATCH** **Bilan quotidien** **24 h**

Relire un lot complet pour recalculer les tendances et les bilans.

**STREAMING** **Alerte** **< 5 s**

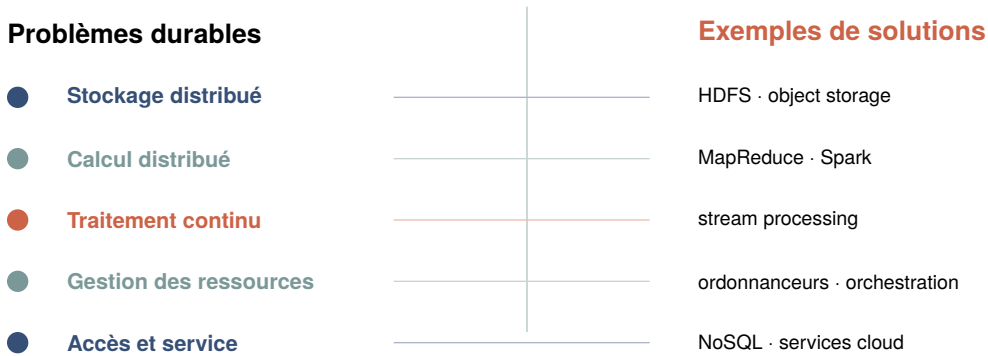
Traiter les événements en continu ; contrôler retard, doublons et reprise.

**Le mode dépend du délai acceptable ; un même système peut combiner batch et streaming.**

# Les outils changent. Les problèmes restent.



**Les technologies évoluent ; les responsabilités d'architecture restent.**




**Volume, débit, latence, disponibilité, qualité et coût restent les questions fondamentales.**

# L'IA à l'échelle dépend des données



**Un modèle utile dépend de données préparées, accessibles et surveillées.**

### DONNÉES RÉELLES

capteurs · logs ·
images · événements

### DATA ENGINEERING

1. **Collecter** — ingestion fiable
2. **Fiabiliser** — qualité · fraîcheur
3. **Préparer** — features · embeddings
4. **Servir** — accès traçable

### IA À L'ÉCHELLE

**ENTRAÎNER · INFÉRER**

modèle disponible et observable

**MONITORING** — qualité des données · dérive du modèle · latence · effet réel

**À grande échelle, la performance du modèle dépend autant de la plateforme de données que du calcul.**

# Activité — quelle architecture défendez-vous ?



ACTIVITÉ · VOTE · JUSTIFICATION

**Choisissez une option et nommez le test qui pourrait vous donner tort.**

Besoin : 486 To planifiés · 50 Mo/s · $p95 < 5$ s · perte d'un nœud tolérée

**A**

## Serveur actuel

20 To · nœud unique

**B**

## Scale-up

machine plus grande

**C**

## Scale-out

plusieurs nœuds

**Défendez une option et indiquez le test qui pourrait l'invalider.**

**Une option reste une hypothèse tant que charge, délai, panne et coût ne sont pas testés.**

# Fermer la boucle — mesurer pour mieux décider



**Une plateforme crée de la valeur lorsque chaque action produit un retour mesurable.**

**Latence $p95$ : 3,8 s** | **82 % d'alertes pertinentes** | **6 faux positifs / jour**

**Apprendre, corriger, améliorer**

**1 Mesurer**
signaux du terrain

**2 Détecter**
anomalie confirmée

**3 Décider**
choix opérationnel

**4 Agir**
maintenance réalisée

**5 Évaluer**
résultat mesuré

**Être data-driven, c'est mesurer l'effet de chaque action pour améliorer la décision suivante.**

# La carte de décision Big Data



**Sept questions permettent de raconter et défendre une architecture.**

**1 Décision**
Quelle action améliorer ?

**2 Mesures**
Volume, débit, délai ?

**3 Qualité**
Donnée fraîche et fiable ?

**4 Mode**
Batch, flux ou les deux ?

**5 Échelle**
Renforcer ou distribuer ?

**6 Résilience**
Quelles pannes tolérer ?

**7 Prouver le résultat et le coût**

**Partir du besoin, mesurer, choisir le moyen le plus simple, puis prouver qu'il tient.**

# Étape suivante — répartir les responsabilités



**Si la distribution est justifiée, qui stocke, qui arbitre et qui calcule ?**

## HDFS

**STOCKER**

Où placer les blocs
et leurs copies ?

## YARN

**ARBITRER**

Qui reçoit CPU
et mémoire ?

## MapReduce

**CALCULER**

Comment distribuer
un calcul batch ?

**Hadoop devient une architecture candidate parce que le besoin a été mesuré — jamais par réflexe.**